AI ENGINEERING

Stop Paying Twice for LLM Prompts

1 Why your LLM apps feel slow and expensive

2 The dead-simple architecture fix: Prompt Caching

SV
Srijan Verma
AI Systems Engineer

SWIPE →

The Bottleneck

1 Your system prompt is identical for thousands of requests.

2 Yet, the model re-processes every single token from scratch each time.

SV

Srijan Verma

AI Systems Engineer

How Prompt Caching Works

1. Store the precomputed KV pairs of static prefixes in memory.
2. Match incoming tokens against the cached prefix.
3. Only compute attention on brand-new user tokens.

SV

Srijan Verma

AI Systems Engineer

The Logic Pattern

1 Reuse past key-values when a prefix hits cache.

main.py

```
if prefix in kv_cache:
    # Skip prefix, only process new tokens
    output = model.generate(new_tokens, past_key_values=kv_cache[prefix])
else:
    output = model.generate(full_tokens)
    kv_cache[prefix] = model.past_key_values
```

SV

Srijan Verma

AI Systems Engineer

Production Impact

1. 50% to 85% reduction in Time-to-First-Token (TTFT).

2. Up to 90% cost drop on shared system instructions and document context.

SV

Srijan Verma

AI Systems Engineer

Rule of Thumb

1. Keep static instructions at the very beginning.
2. Keep dynamic user inputs at the very end.
3. Follow for more practical LLM engineering tips.

SV

Srijan Verma

AI Systems Engineer